

FAST
FORWARD

АКОС

Лекция 9
Сигналы, pipe

Сигналы

Сигнал - число передающееся от ядра процессу

Всего их 32 базовых

Обычно каждому сигналу соответствует “событие” в системе

Передача в виде вызова функции (диспозиция)

Примеры

SIGSEGV - сигнал процессу завершиться, обычно выводит "Segmentation fault"

SIGINT - прерывание с клавиатуры (Ctrl+C)

SIGALRM - прерывание по таймеру выставленному вызовом alarm

SIGUSR1 - зарезервировано на усмотрение пользователя

Диспозиция

По умолчанию одна из:

Term - завершить процесс

Ign - игнорировать

Core - Term + записать память процесса на диск

Stop - приостановить процесс

Cont - возобновить приостановленный процесс

Сигналы завершения

- SIGINT** - Term - Ctrl+C от пользователя
- SIGTERM** - Term - Вежливое
- SIGKILL** - Term - Принудительное, нельзя поменять обработку
- SIGABRT** - Core - Аварийное вызванное самой программой
- SIGSEGV** - Core - Ошибка доступа к памяти

Другие интересные

SIGCHLD - Ign - Остановка завершения / продолжение дочернего процесса

SIGSTOP - Stop - Приостановка процесса

SIGCONT - Cont - Возобновление процесса

SIGILL - Term - Невалидная инструкция

Кастомные обработчики

```
typedef void (*sighandler_t)(int)  
sighandler_t signal(int signum, sighandler_t handler)
```

На все кроме SIGKILL и SIGSTOP

Можно передать SIG_IGN или SIG_DFL

SystemV - сбросить после обработки

BSD - оставить

Linux - BSD стиль

Пример

```
void sayhi(int signo) {  
    write(STDOUT_FILENO, "hi\n", 3);  
}  
  
int main() {  
    signal(SIGINT, sayhi);  
    while (read(...)) {  
        write(...);  
    }  
}
```


Пример для SystemV

```
void sayhi(int signo) {  
    write(STDOUT_FILENO, "hi\n", 8);  
    signal(signo, sayhi);  
}  
  
int main() {  
    signal(SIGINT, sayhi);  
    while (read(...)) {  
        write(...);  
    }  
}
```

Нюансы

Сигнал может прийти в любой момент времени

При поступлении выполнение программы останавливается

Продолжается после выхода из сигнала

Стек - стек программы + red zone 128 байт

Во время сигнала второго прийти не может

SIGCHLD

Дефолтно - Ignore

Явно выставить SIG_IGN - не нужен wait

sigaction

```
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

void (*sa_handler)(int) - такой же, как в signal

int sa_flags - маска флагов

- SA_RESTART
- SA_RESETHAND

Пример

```
int main() {  
    struct sigaction sa = {  
        .sa_handler = sayhi,  
        .sa_flags = SA_RESTART,  
    };  
    sigaction(SIGINT, &sa, NULL);  
    while (read(...)) {  
        write(...);  
    }  
}
```

Маски pending blocked

На каждый сигнал два бита - есть ли входящий, заблокирован ли

При получении

- 1) Если заблокирован - выставили пендинг
- 2) Если уже обрабатывается другой - пендинг
- 3) Иначе - сразу запускаем
- 4) После обработки проверяем маски

Отправка сигнала

```
int kill(pid_t pid, int sig);
```

Ради безопасности - только процессам пользователя

pid = -1 – всем кому возможно

Блокировка сигналов

```
int sigprocmask(int how, const sigset_t * _Nullable restrict set,  
                sigset_t * _Nullable restrict oldset);
```

SIG_BLOCK - добавить

SIG_UNBLOCK - удалить

SIG_SETMASK - выставить

set = NULL - трюк получить текущую



FAST
FORWARD

sigset_t

```
int sigemptyset(sigset_t *set);           // пустой
int sigfillset(sigset_t *set);           // полный
int sigaddset(sigset_t *set, int signum); // добавить
int sigdelset(sigset_t *set, int signum); // убрать
int sigismember(const sigset_t *set, int signum); // проверить
```

Пример

```
int main() {  
    ...  
    sigset_t set;  
    sigemptyset(&set);  
    sigaddset(&set, SIGUSR1);  
    sigprocmask(SIG_SETMASK, &mask, NULL);  
    pause();  
}
```

Пример

```
int main() {  
    ...  
    sigset_t set;  
    sigemptyset(&set);  
    sigaddset(&set, SIGUSR1);  
    sigprocmask(SIG_SETMASK, &mask, NULL);  
    sigsuspend(&mask);  
}
```

Проблемы с синхронизацией

```
int ctr = 0;
```

```
void handler(int signum) {
```

```
    ++ctr;
```

```
}
```

```
int main() {
```

```
    while (1) {
```

```
        printf("%d\n", ctr);
```

```
    }
```

```
}
```

Проблемы с синхронизацией

```
volatile sig_atomic_t ctr = 0;
```

```
void handler(int signum) {
```

```
    ++ctr;
```

```
}
```

```
int main() {
```

```
    while (1) {
```

```
        printf("%d\n", ctr);
```

```
    }
```

```
}
```

Проблемы с синхронизацией

```
volatile sig_atomic_t ctr = 0;
```

```
void handler(int signum) {  
    printf("%d\n", signum); ++ctr;  
}
```

```
int main() {  
    while (1) {  
        printf("%d\n", ctr);  
    }  
}
```

Продвинутый обработчик

```
void (*sa_sigaction)(int, siginfo_t*, void*);
```

```
struct siginfo_t {  
    int    si_trapno; // hardware-generated signal  
    pid_t  si_pid;    // Sending process ID  
    void   *si_addr;  // Memory location caused fault  
    ...  
}
```

Сигналы реального времени

Другие номера - от SIGRTMIN до SIGRTMAX

Не используются системой

Повторные не теряются

Можно отправить вместе с сигналом дополнительно 8 байт

```
int sigqueue(pid_t pid, int sig, const union sigval value);
```


Канал (pipe)

```
int pipe(int pipefd[2]);
```

pipefd[0] - читать

pipefd[1] - писать

Канал (pipe)

`read(...):`

- когда есть данные в буфере - возвращает эти данные
- когда данных нет, но существует "писатель" - блокируется и ждёт записи или закрытия писателей
- когда данных и писателей нет, возвращает 0 - EOF

`write(...):`

- когда в буфере есть место для данных - записывает их туда
- когда места нет, но существует "читатель" - блокируется и ждёт освобождения буфера или закрытия читателей
- когда читателей нет, присылает пишущему процессу сигнал SIGPIPE

Пример

```
pid_t pid = fork();
```

```
int fd[2];
```

```
pipe(fd);
```

```
if (pid == 0) {
```

```
    dup2(fd[0], 0);
```

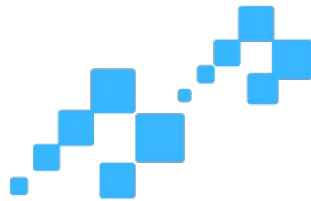
```
    ...
```

```
} else {
```

```
    dup2(fd[1], 1);
```

```
    ...
```

```
}
```



FAST
FORWARD

AKOC

QCourse.ru 2026